Database Management System

Course Code: **2114112**

Module1

Shilpa Ingoley

Course Code	Course Name	Teaching Scheme (Contact Hours)			Credits Assigned			
		Theory	Pract.	Tut.	Theory	Pract.	Tut.	Total
2114112	Database Management System	3	2	-	3	1	-	4

Course Code	Course Name	Theory					Term work	Pract / Oral	Total
		Internal Assessment			End Sem Exam	Exam Duration (in Hrs)			
		Test 1	Test 2	Avg.					
2114112	Database Management System	20	20	40	60	2	--	--	100

Rationale:

Today's data-driven world, Database Management Systems (DBMS) are essential for efficiently storing, managing, and analyzing data. This course equips students with foundational concepts and practical skills to design and implement robust data-driven solutions across diverse domains.

Sr. No.	Course Objectives:
1	To Understand the fundamentals of a database systems
2	Develop entity relationship data model /EER and its mapping to relational model
3	Learn relational algebra and Formulate SQL queries.
4	Apply normalization techniques to normalize the database
5	Understand concept of transaction, concurrency control and recovery techniques
6	Explore and understand recent databases and their application

Sr No	Course Outcomes:	BL
CO1	Understand concepts of DBMS and design ER/EER diagram for real world application.	L2, L3
CO2	Apply mapping rules to construct relational model from data model and formulate relational algebra queries.	L3
CO3	Apply SQL queries for database operations.	L3
CO4	Analyze and apply normalization techniques to relational database design.	L3, L4
CO5	Understand transaction, concurrency and recovery techniques to analyze conflicts in multiple transactions.	L2
CO6	Understand recent databases.	L2

Syllabus:

DETAILED SYLLABUS:

Sr. No.	Name of Module	Detailed Content	Hours	CO Mapping
0	Prerequisite	Basic knowledge of Data structure, Fundamentals of computer system		
I	**Introduction to Database and Data Modeling**	**Introduction:** Definitions and application, Characteristics of databases, DBMS architecture, ACID Properties **The Entity-Relationship (ER) Model**: Entity types: Weak and strong entity sets, Entity sets, Types of Attributes, Keys, Relationship constraints: Cardinality and Participation, Extended Entity-Relationship (EER) Model: Generalization, Specialization and Aggregation **Self-learning Topics:** Design an ER model for any real time case study.	**08**	**CO1**
II	**Relational Model and Relational Algebra**	Introduction to the Relational Model, relational schema and concept of keys. Mapping the ER and EER Model to the Relational Model, Relational Algebra- operators (Selection(σ),Projection(π),Union(U),Difference (−),CartesianProduct(×),Join(⋈), Intersection (∩),Rename (ρ)), Relational Algebra Queries **Self-learning Topics:** Practice writing queries to perform common database tasks (e.g., selecting data, joining tables	**05**	**CO2**

III	**Structured Query Language (SQL)**	Overview of SQL, Data Definition Commands, Integrity constraints: key constraints, Domain Constraints, Referential integrity, check constraints, Data Manipulation commands, Data Control commands, Transaction Control Commands. aggregate function-group by, having, order by, joins, Nested and complex queries, Views in SQL, Set and string operations, Triggers, Introduction to Pl/SQL Block Structure **Self-learning Topics:** LeetCode (SQL practice problems), HackerRank (SQL challenges)	**10**	**CO3**
IV	**Database Normalization**	Pitfalls in relational database designs, Concept of normalization, Function Dependencies, FD closure, First Normal Form, 2NF, 3NF, BCNF, 4NF. **Self-learning Topics:** Consider any real time application and normalization upto 3NF/BCNF	**5**	**CO4**
V	**Transaction Management and Concurrency Control**	Transaction concept, Transaction states, Concurrent Executions, Serializability-Conflict and View, Concurrency Control: Lock-based, Timestamp-based protocols, Recovery System: log-based recovery, Introduction to Deadlock handling **Self-learning Topics:** SQL challenges related to transactions and concurrency	**7**	**CO5**

VI	**Introduction to Modern databases**	Recent trends in the industry, Introduction of Cloud Database, Introduction of Distributed Database, Introduction to NOSQL Database and Object-Oriented Databases **Self-learning Topics:** Learn about emerging database technologies. Explore different NoSQL types. Learn how object-oriented programming concepts like objects and inheritance are applied to database management systems.	**4**	**CO6**

Text Books:

1. Elmasri and Navathe, Fundamentals of Database Systems, 7thEdition, Pearson Education
2. Korth, Slberchatz, Sudarshan, Database System Concepts, 7thEdition, McGraw Hill
3. Raghu Ramkrishnan and Johannes Gehrke, Database Management Systems, TMH
4. RajkumarBuyya, Christian Vecchiola, S ThamaraiSelvi, "Mastering Cloud Computing", Tata McGraw-Hill Education

Reference Books:

1. Peter Rob and Carlos Coronel, Database Systems Design, Implementation and Management, Thomson Learning, 5th Edition.
2. Dr. P.S. Deshpande, SQL and PL/SQL for Oracle 10g, Black Book, Dreamtech Press.
3. G. K. Gupta, Database Management Systems, McGraw Hill, 2012

Online References:

Sr. No.	Website Name
1.	NPTEL Lecture Series: Database Management system By Prof. Partha Pratim Das, Prof. Samiran Chattopadhyay | IIT Kharagpur
2.	https://www.classcentral.com/course/swayam-database-management-system-9914
3.	https://www.mooc-list.com/tags/dbms
4.	W3Schools: SQL tutorials

Internal Assessment (IA) for 40 marks:

IA will consist of Two Compulsory Internal Assessment Tests. Approximately 40% to 50% of syllabus content must be covered in First IA Test and remaining 40% to 50% of syllabus content must be covered in Second IA Test.

End Semester Internal Examination for 40 marks:

Question paper format:

- Question Paper will comprise of a total of **six questions each carrying 20 marksQ.1** will be **compulsory** and should **cover maximum contents of the syllabus**
- **Remaining questions** will be **mixed in nature** (part (a) and part (b) of each question must be from different modules. For example, if Q.2 has part (a) from Module 3 then part (b) must be from any other Module randomly selected from all the modules)
- A total of **Three questions** needs to be answered.

Module 1: Introduction Database Concepts

Syllabus:

Introduction: Definitions and application, Characteristics of databases, DBMS architecture, ACID Properties

The Entity-Relationship (ER) Model: Entity types: Weak and strong entity sets, Entity sets, Types of Attributes, Keys, Relationship constraints: Cardinality and Participation, Extended Entity-Relationship (EER) Model: Generalization, Specialization and Aggregation

Module 1: Introduction Database Concepts

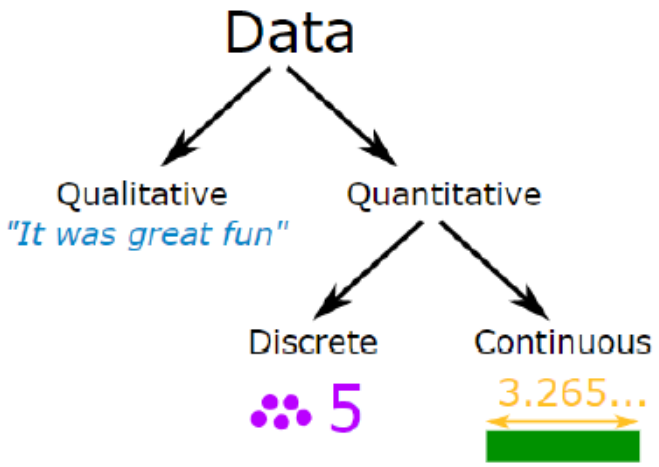
- 1. Introduction
- 2. Definitions and application
- 3. Characteristics of databases
- 4. DBMS architecture
- 5. ACID Properties

Introduction

- Modern enterprises have data that need to store in ways which will be easy to retrieve later
- Common database: list of names and addresses of people who deal with enterprise
  - For smaller business, data can be maintained on paper, word processor or spreadsheet
  - For larger business, database technology is needed
- Database management is about managing information resources of an enterprise

Introduction

- Data is a collection of facts, such as numbers, words, measurements, observations or just descriptions of things.



- Data in raw or unorganized form (such as alphabets, numbers, or symbols) that refer to, or represent, conditions, ideas, or objects. Data is limitless and present everywhere in the universe

Database

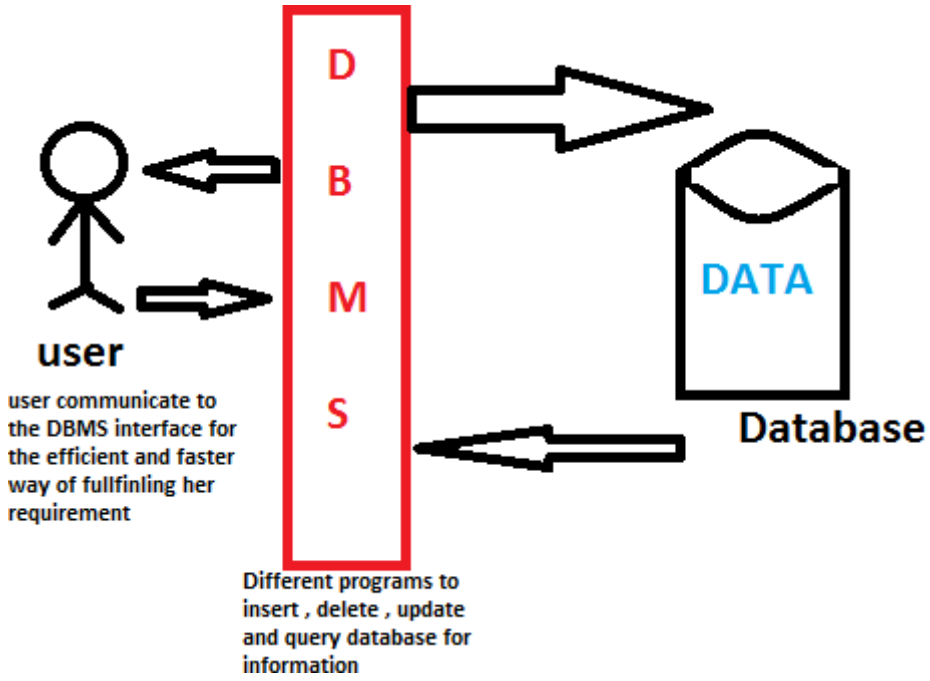
- Organized collection of data
- Easily accessed, managed and updated
- Example:
  - Collage database stores information about students, teachers, classes, subjects, etc...

First Name	Last Name	Address	City	Age
Mickey	Mouse	123 Fantasy Way	Anaheim	73
Bat	Man	321 Cavern Ave	Gotham	54
Wonder	Woman	987 Truth Way	Paradise	39
Donald	Duck	555 Quack Street	Mallard	65
Bugs	Bunny	567 Carrot Street	Rascal	58
Wiley	Coyote	999 Acme Way	Canyon	61
Cat	Woman	234 Purrfect Street	Hairball	32
Tweety	Bird	543	Itotltaw	28

Records

DATABASE MANAGEMENT SYSTEM

A database management system is a software used to perform different operations, like addition, access, updating, and deletion of the data



Need of DBMS

- When dealing with huge amount of data, there are two things that require optimization:
  - **Storage of data**

    eg: saving & salary account in same bank
  - **Retrieval of data**.

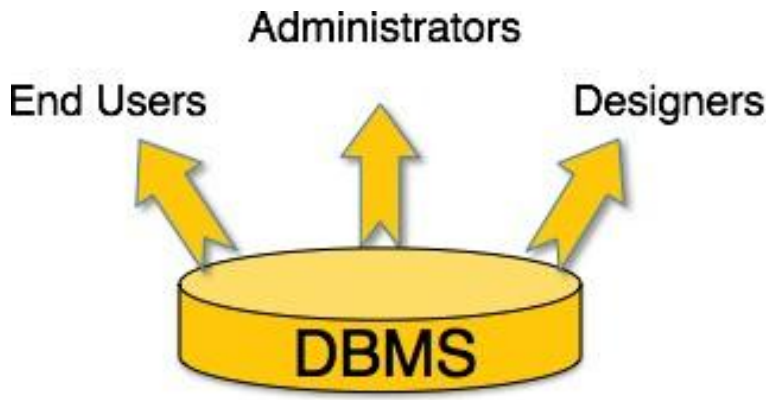
- Managing the data – eg: university
  - Add
  - Delete
  - Update
  - Retrieve

Why do we use DBMS?

- Data independence and efficient access.
- Reduced application development time.
- Data integrity and security. Different users may access different data subsets.
- Uniform data administration.
- Concurrent access, recovery from crashes.

Users of DBMS

Component Name	Task
Application Programmers	The Application programmers write programs in various programming languages to interact with databases.
Database Administrators	Database Admin is responsible for managing the entire DBMS system. He/She is called Database admin or DBA.
End-Users	The end users are the people who interact with the database management system. They conduct various operations on database like retrieving, updating, deleting, etc.



History of Database Systems

1950s and early 1960s:

- **Magnetic tapes** were developed for data storage
- Data processing tasks such as payroll were automated, with data stored on **tapes.**
  - Data could also be input from punched card decks, and output to printers.
- Late 1960s and 1970s: The use of **hard disks** in the late 1960s changed the scenario for data processing greatly, since hard disks allowed direct access to data.
- With disks, network and hierarchical databases could be created that allowed data structures such as lists and trees to be stored on disk. Programmers could construct and manipulate these data structures.
- In the 1970's the EF CODD defined the **Relational Model.**

History of Database Systems(contd..)

In the 1980's:

- Initial commercial relational database systems, such as **IBM DB2, Oracle, Ingress, and DEC Rdb**, played a major role
- In the early 1980s, relational databases had become competitive with network and hierarchical database systems even in the area of performance.
- The 1980s also saw much research on **parallel and distributed databases**, as well as initial work on **object-oriented databases.**

History of Database Systems(contd..)

Early 1990s:

- The **SQL language** was designed primarily for the transaction processing applications.
- **Decision support and querying** were seen as a major application area for databases.
- Database vendors also began to add object-relational support to their databases.

Late 1990s:

- The major event was the explosive growth of the World Wide Web.
- Database systems now had to support very high transaction processing rates, as well as very high reliability and 24 * 7 availability
- Database systems also had to support Web interfaces to data.

Evolution of Database Systems

- **File Management System**
- **Hierarchical database System**
- **Network Database System**
- **Relational Database System**

File Management System

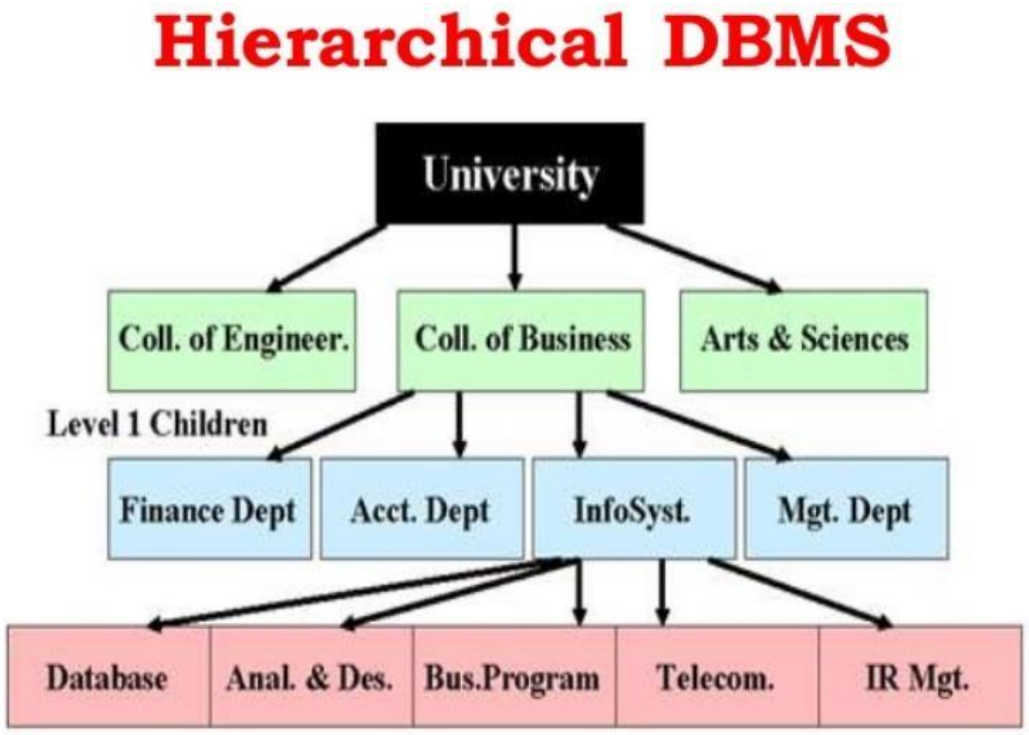
- All data is stored on a single large file
- Disadvantage :
  - searching a record or data takes a long time
  - updating or modifications to the data cannot be handled easily
  - sorting the records took long time and so on
- All these drawbacks led to the introduction of the Hierarchical Database System.

Hierarchical Database System

- Advantage: FMS drawback of accessing records and sorting records which took a long time was **removed by parent-child relationship between records in database**.
- Drawback: any modification or addition made to the structure then the whole structure needed alteration which made the task a tedious one.

Fig: Hierarchical Database System

Hierarchical database

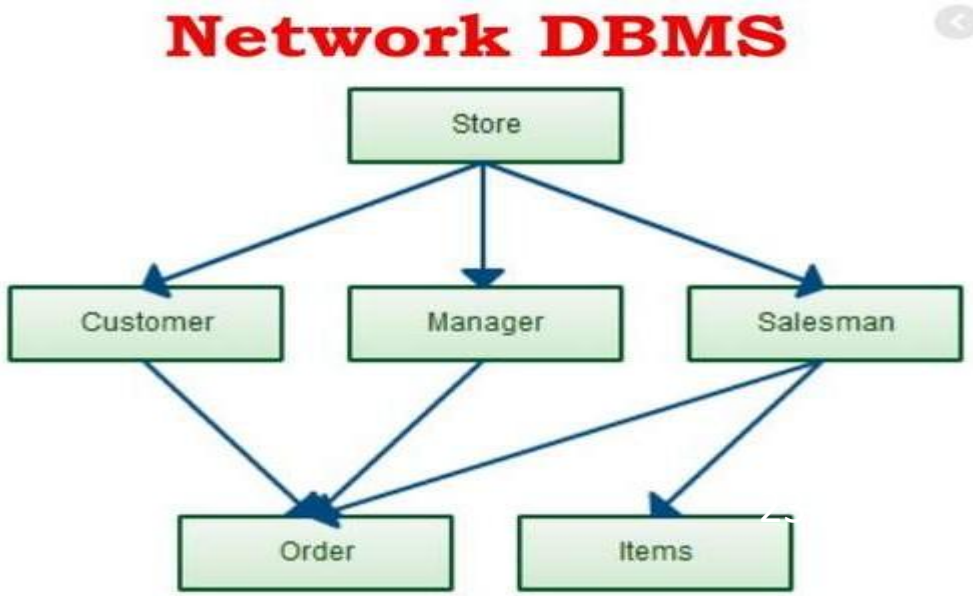


- It represents the data in a tree-like structure in which there is a single parent for each record.
- Data get stored in the form of records that are connected via links.
- This model structure allows the one-to-one and a one-to-many relationship between two/ various types of data.
- This structure is very helpful in describing many relationships in the real world; table of contents, any nested and sorted information.

Network Database System

- In this the main concept of **many-many relationships got introduced**.
- But this also followed the same technology of pointers to define relationships with a difference in this made in the introduction if grouping of data items as sets.

- Representation of data is in the form of nodes connected via links between them.
- Unlike the hierarchical database, it allows each record to have multiple children and parent nodes to form a generalized graph structure.

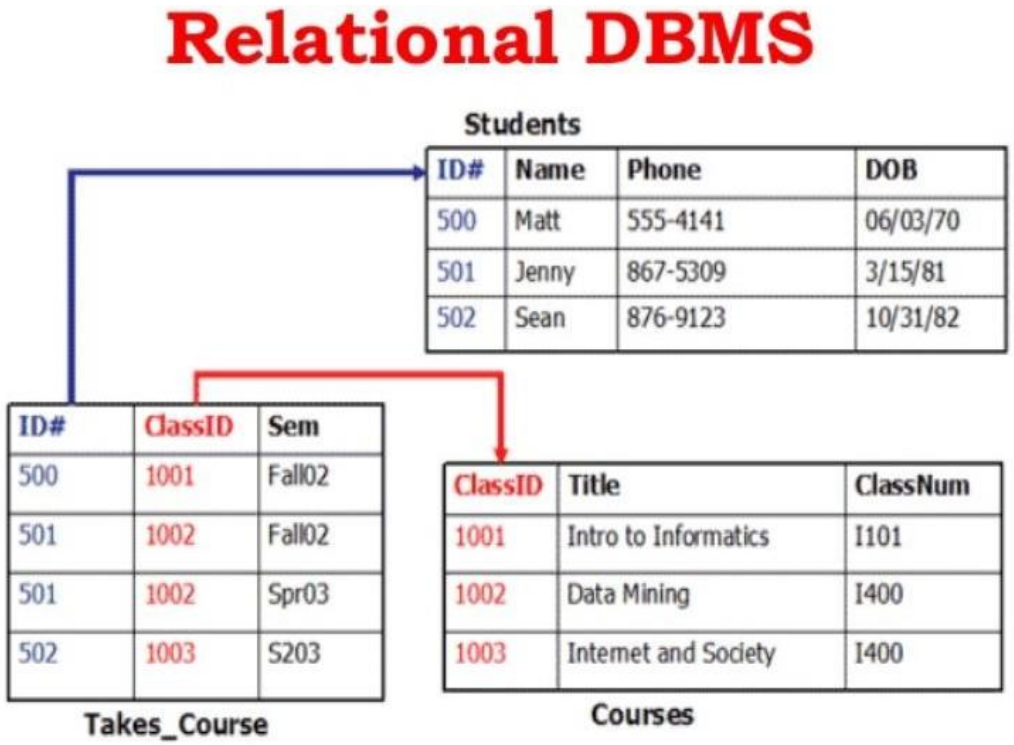


Relational Database System

- In order to overcome all the drawbacks of the previous systems, the Relational Database System got introduced in which **data get organized as tables and each record forms a row with many fields or attributes in it**.
- Relationships between tables are also formed in this system.

Name	FName	City	Age	Salary
Smith	John	3	35	$280
Doe	Jane	1	28	$325
Brown	Scott	3	41	$265
Howard	Shemp	4	48	$359
Taylor	Tom	2	22	$250

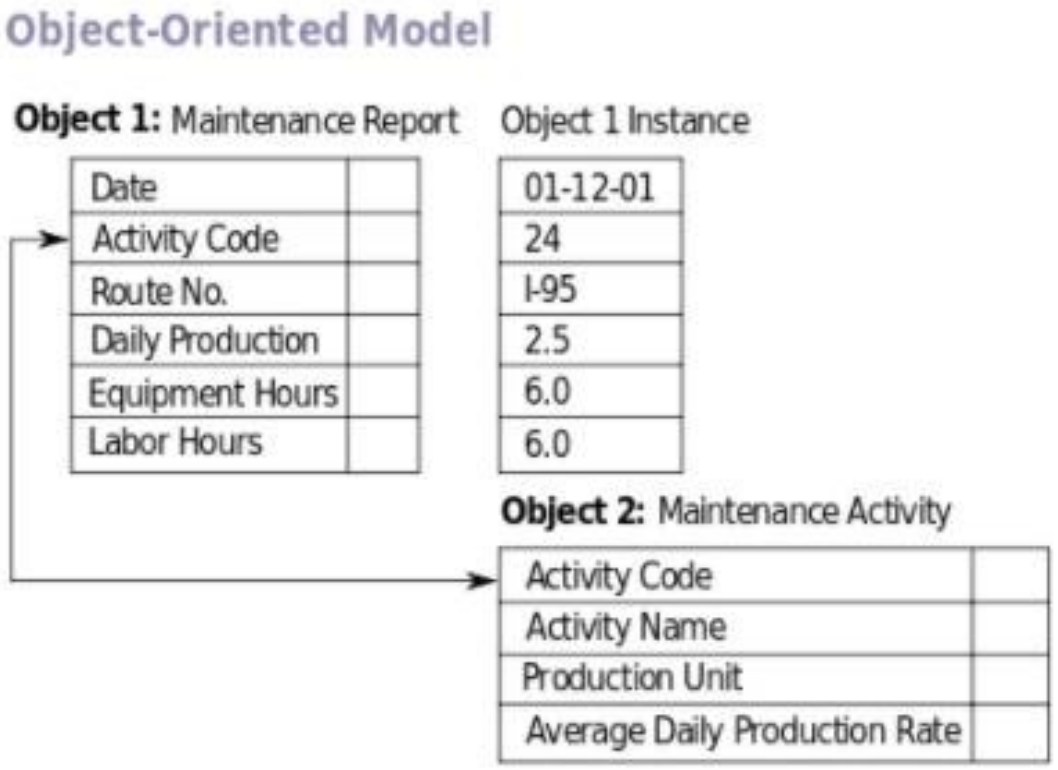
Relational database



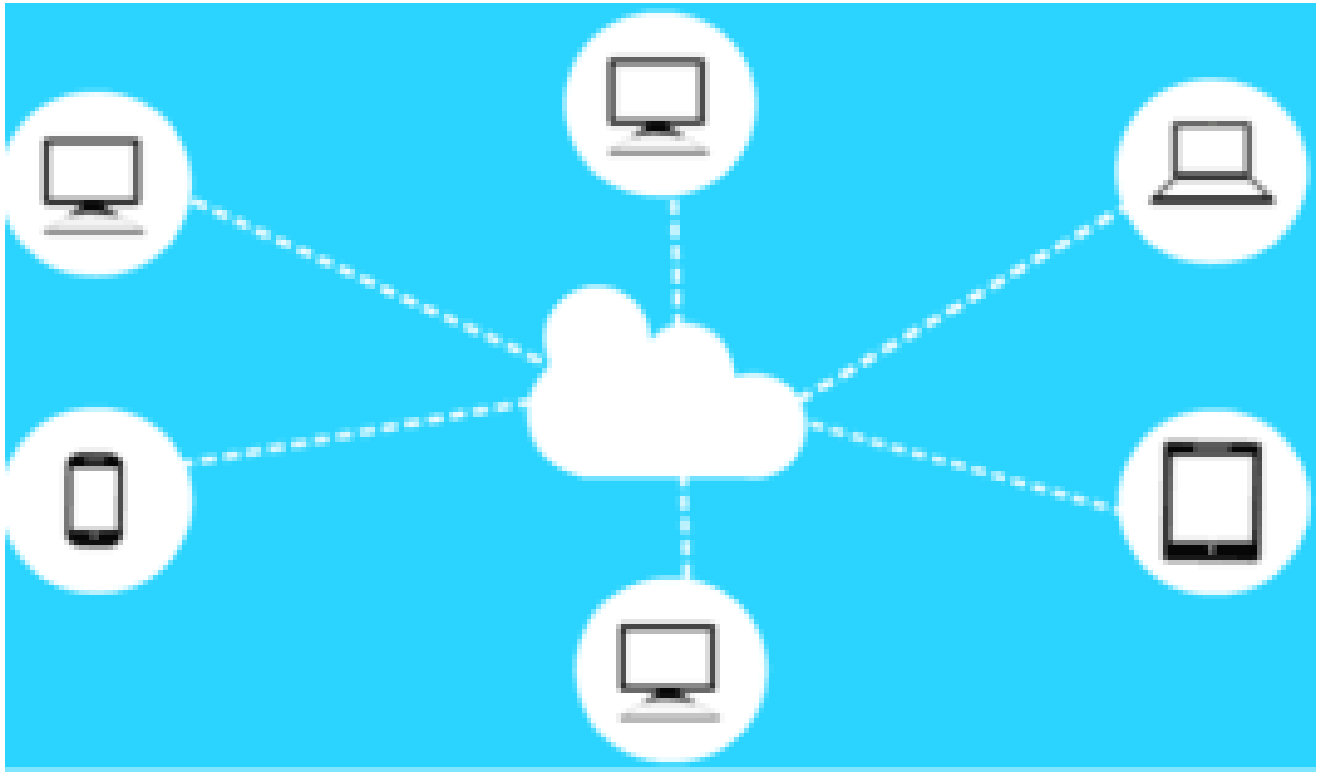
- Stores data in the form of rows(tuple) and columns(attributes), and together forms a table(relation).
- It uses SQL for storing, manipulating, as well as maintaining the data.

Object-oriented database

- It uses the object-based data model approach for storing data in the database system.
- The data is represented and stored as objects which are similar to the objects used in the object-oriented programming language.



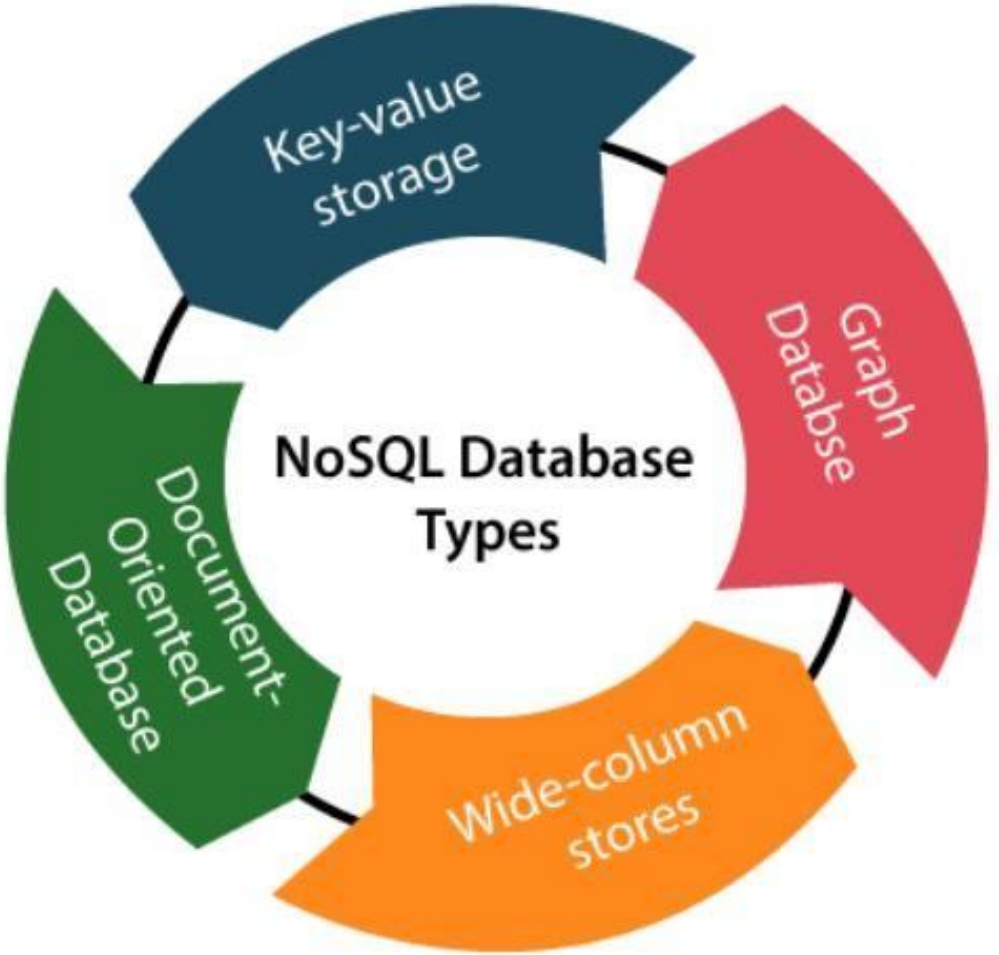
Cloud database



- Data is stored in a virtual environment and executes over the cloud computing platform.
- It provides users with various cloud computing services (SaaS, PaaS, IaaS, etc.) for accessing the database.

NoSQL database

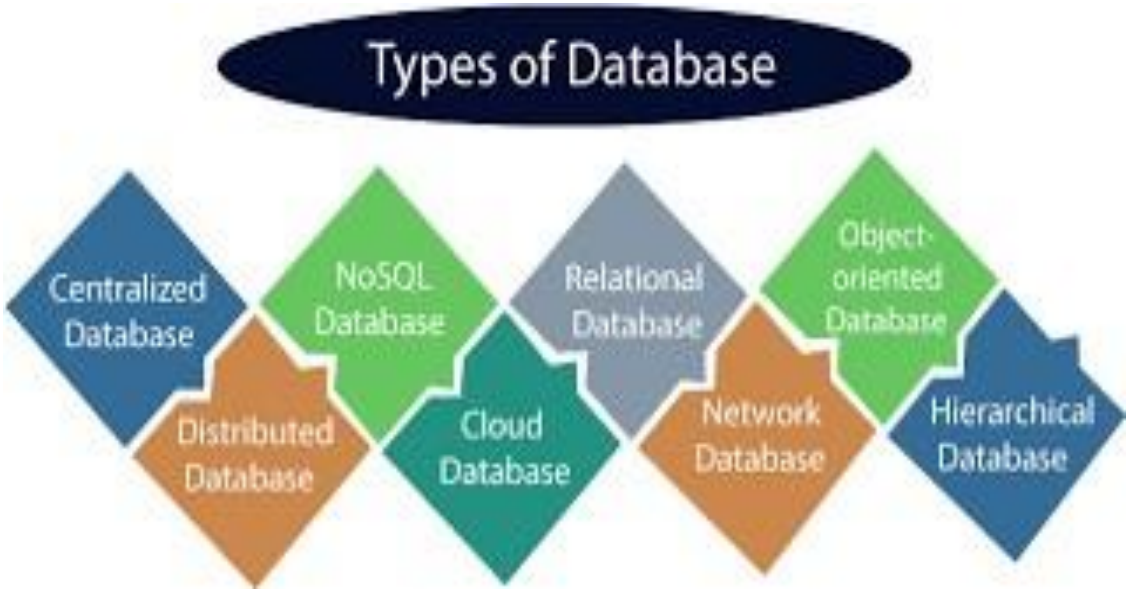
- Used for storing a wide range of data sets.
- It came into existence when the demand for building modern applications increased.
- It presented a wide variety of database technologies in response to the demands.
- NoSQL database is further divided into four types



DBMS - Most Popular Database Management Systems

Different Types of Databases

- Traditional Applications:
  - Numeric and Textual Databases
- More Recent Applications:
  - Multimedia Databases
  - Geographic Information Systems (GIS)
  - Data Warehouses
  - Distributed Database
  - Real-time Databases



2 Application of DBMS

- **1.Airline and railways:** Airlines and railways use online database for reservation and for displaying the schedule information.
- **2.Banking:** Banks use databases for customer inquiry, account, loans and other transactions.
- **Education:** schools and college use database for course registration results and other information.
- **4.Telecommunications:** Telecommunication departments use databases to store information about the communication network, telephone numbers, record of calls,for generation monthly bills. Etc.
- **5.Credit card transaction:** Database are used for keeping track of purchases on credit cards in order to generate monthly statements.
- **6.E-Commerce:** Integration of heterogeneous information source for business activity such as online shopping, booking of holiday package,consulting a doctor etc.
- **7.Health cares are information system and electronics patient record**: Database are used for maintaining the patient health care details.
- **8.Digital Libraries and digital publishing:** Database are used for management and delivery of large bodies of textual and multimedia data.
- **9.Finance:** Databases are used for storing information such as sales, purchase of stocks and bonds or data useful for online trading.
- **10.Sales:** database is used to store product, customer and transaction details.
- **11.Human resources:** Organization use database for storing information about their employee, salaries, benefits, taxes and for generating salary checks.

3. Characteristics of databases

Main Characteristics of the Database System Approach

1. Self-describing nature of a database system:

- A DBMS **catalog** stores the description of a particular database (e.g. data structures, types, and constraints)
- The description is called **meta-data**.
- This allows the DBMS software to work with different database applications.

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

Example of a simplified database catalog

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....	....	.....
....	....	.....
....	....	.....
Prerequisite_number	XXXXNNNN	PREREQUISITE

Note: Major_type is defined as an enumerared type with all known majors. XXXXNNNN is used to define a type with four alpha characters followed by four digits

Figure 1.3
An example of a database catalog for the database in Figure 1.2.

Main Characteristics of the Database System Approach (contd..)

2. Insulation between programs and data:

- The structure of data files is stored in the DBMS catalog separately from the access programs, call this property as **program-data independence**
- Allows changing data structures and storage organization without having to change the DBMS access programs.

3. Data Abstraction:

- A **data model** is used to hide storage details and present the users with a conceptual view of the database.
- Programs refer to the data model constructs rather than data storage details

Main Characteristics of the Database System Approach (contd..)

4. Support of multiple views of the data:

- Each user may see a different view of the database, which describes **only** the data of interest to that user.

TRANSCRIPT

Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

(a)

COURSE_PREREQUISITES

Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

(b)

Main Characteristics of the Database System Approach (contd..)

5. Sharing of data and multi-user transaction processing:

- *Concurrency control* within the DBMS guarantees that each **transaciton** is correctly executed or aborted
- Allowing a set of **concurrent users** to retrieve from and to update the database.
- *Recovery* subsystem ensures each completed transaction has its effect permanently recorded in the database
- **OLTP** (Online Transaction Processing) is a major part of database applications. This allows hundreds of concurrent transactions to execute per second.

Database Management System

- A **database-management system**(DBMS) is a collection of interrelated data and a set of programs to access those data.
- General purpose software system

Goals of DBMS:

primary goal: **provide a way to store and retrieve database information that is both *convenient* and *efficient***

1. Manage large bodies of information
2. Secure information against system failure or tampering
3. Permit data to be shared among multiple users

Simplified Database System Environment

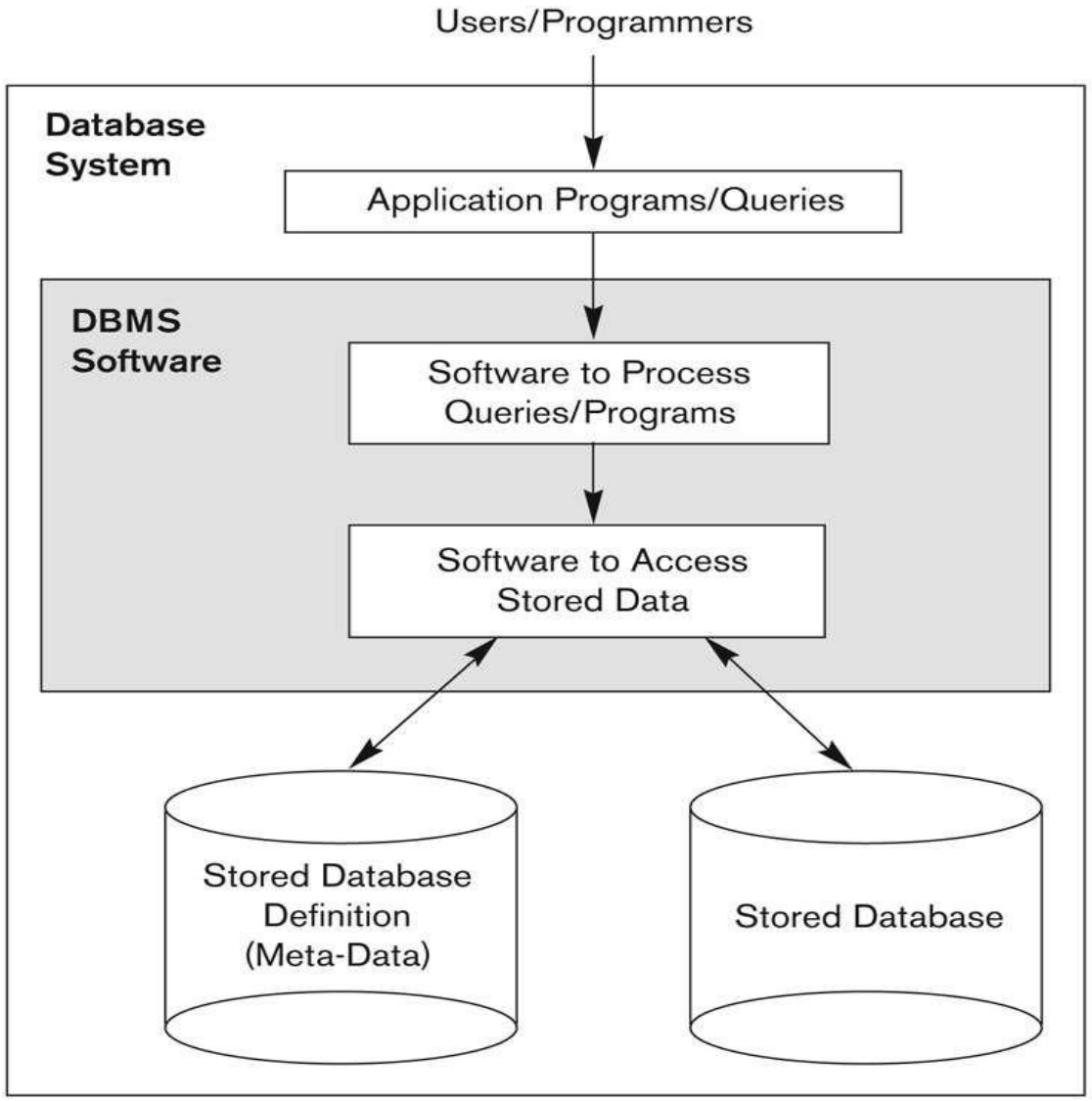


Figure 1.1
A simplified database system environment.

File System

- Computer systems were used to process business records and produce information which were **faster and more accurate than equivalent manual systems**.
- Stored groups of records in separate files, and so they were called **file processing systems.**

1. Collection of data. Any management with the file system, user has to write the procedures
2. Gives the details of the data representation and Storage of data
3. Storing and retrieving of data cannot be done efficiently
4. Concurrent access to the data in the file system has many problems
5. Doesn’t provide crash recovery mechanism
6. Protecting a file under file system is very difficult

Example:

- Consider part of a savings-bank enterprise that stores information about all customers and savings accounts in operating system files. To allow users to manipulate the information, the system has a number of application programs that manipulate the files, including:
- A program to debit or credit an account
- A program to add a new account
- A program to find the balance of an account
- A program to generate monthly statements

  - **For example, suppose that the savings bank decides to offer checking accounts.**

Drawbacks of File System

- **Data Redundancy and Inconsistency**
- **Difficulty in Accessing Data**
- **Data Isolation**
- **Integrity Problems**
- **Atomicity Problem**
- **Concurrent Access Anomalies**
- **Security Problems (ex:**payroll personnel)

Advantages of DBMS over File System

- Controlling redundancy in data storage and in development an maintenance efforts
    - Sharing of data among multiple users
    - Restricting unauthorized access to data.
- Providing persistent storage for program Objects
- Providing Storage Structures (e.g. indexes) for efficient Query Processing
- Providing backup and recovery services.
- Providing multiple interfaces to different classes of users.
- Representing complex relationships among data.
- Enforcing integrity constraints on the database.

File system v/s Database system

File Processing System	DBMS
File system is a collection of data. Any management with the file system, user has **to write the procedures**	DBMS is a collection of data and user is **not required to write the procedures** for managing the database.
File system gives the **details of the data representation and Storage** of data.	DBMS provides **an abstract view** of data that hides the details.
In File system storing and retrieving of **data cannot be done efficiently**.	**DBMS is efficient** to use since there are wide varieties of sophisticated techniques to store and retrieve the data.
Concurrent access to the data in the file system has **many problems** like : Reading the file while deleting some information, updating some information	DBMS takes care of **Concurrent access using some form of locking.**
File system **doesn't provide crash recovery.** Eg. While we are entering some data into the file if System crashes then content of the file is lost	DBMS has **crash recovery mechanism**, DBMS protects user from the **effects of system failures.**
Protecting a file under file system is very **difficult**.	**DBMS has a good protection** mechanism.

Drawbacks of using file systems to store data:

- **Data redundancy and inconsistency**
  - Multiple file formats, duplication of information in different files
- **Difficulty in accessing data**
  - Need to write a new program to carry out each new task
- **Data isolation** — multiple files and formats
- **Integrity problems**
  - Integrity constraints (e.g. account balance > 0) become part of program code
  - Hard to add new constraints or change existing ones

Data abstraction and data Independence

Data abstraction

- Database systems are made-up of complex data structures.
- To ease the user interaction with database, the **developers hide internal irrelevant details** from users.
- **This process of hiding irrelevant details from user is called data abstraction.**
- There are three levels of abstraction:

1. **Physical** – describes how a record is stored
2. **Logical** – describes data stored in a database & the relationships among the data
3. **View** – describes the user interaction with database

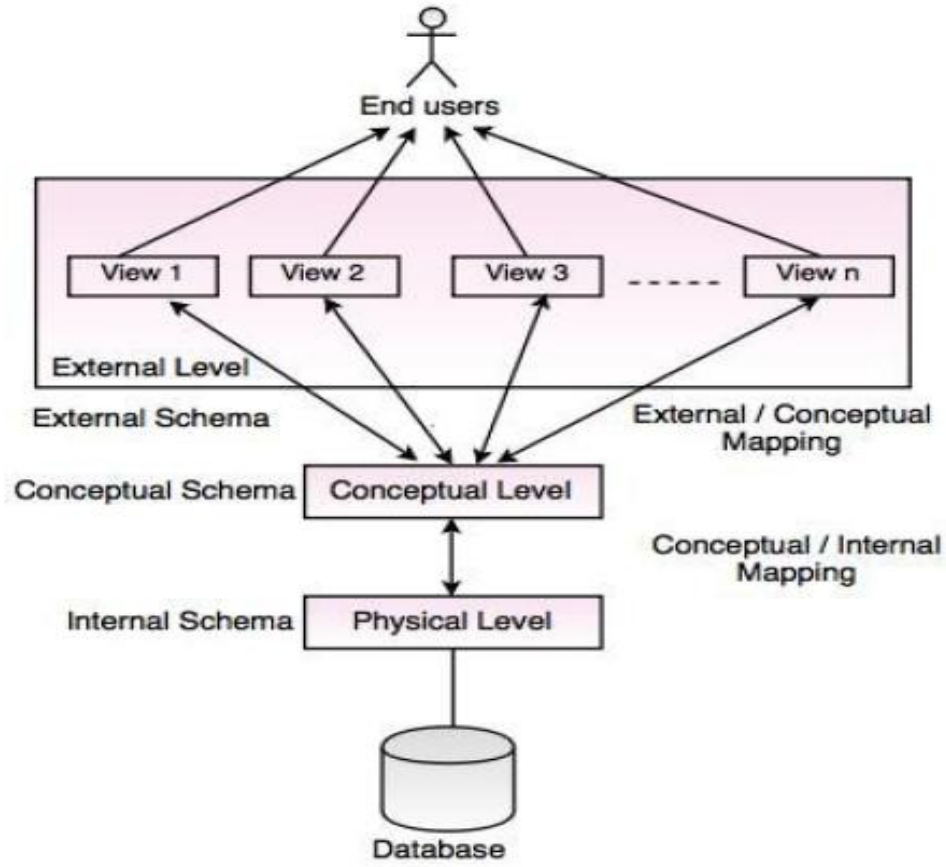


Fig. Three Level Architechture of DBMS

THREE LEVEL ARCHITECTURE OF DATABASE SYSTEM

Example

Let’s say we are storing customer information in a customer table.

1. **Physical -** these records can be described as blocks of storage (bytes, gigabytes, terabytes etc.) in memory. These details are often hidden from the programmers.

2. **Logical** - these records can be described as fields and attributes along with their data types, their relationship among each other can be logically implemented. The programmers generally work at this level because they are aware of such things about database systems.

3. **View** - user just interact with system with the help of GUI and enter the details at the screen, they are not aware of how the data is stored and what data is stored; such details are hidden from them.

Data Independence

- The main purpose of data abstraction is achieving data independence in order to save time and cost required when the database is modified or altered.

- There are two levels of data independence:
  1. **Physical level data independence**
  2. **Logical level data independence**

Physical level data independence

- Characteristic of being able to **modify the physical schema without any alterations to the conceptual or logical schema**.
- Conceptual structure of the database would not be affected by any change in storage size of the database system server.
- These alterations or modifications to the physical structure may include:
  - **Utilising new storage devices.**
  - **Modifying data structures used for storage.**
  - **Altering indexes or using alternative file organisation techniques etc.**

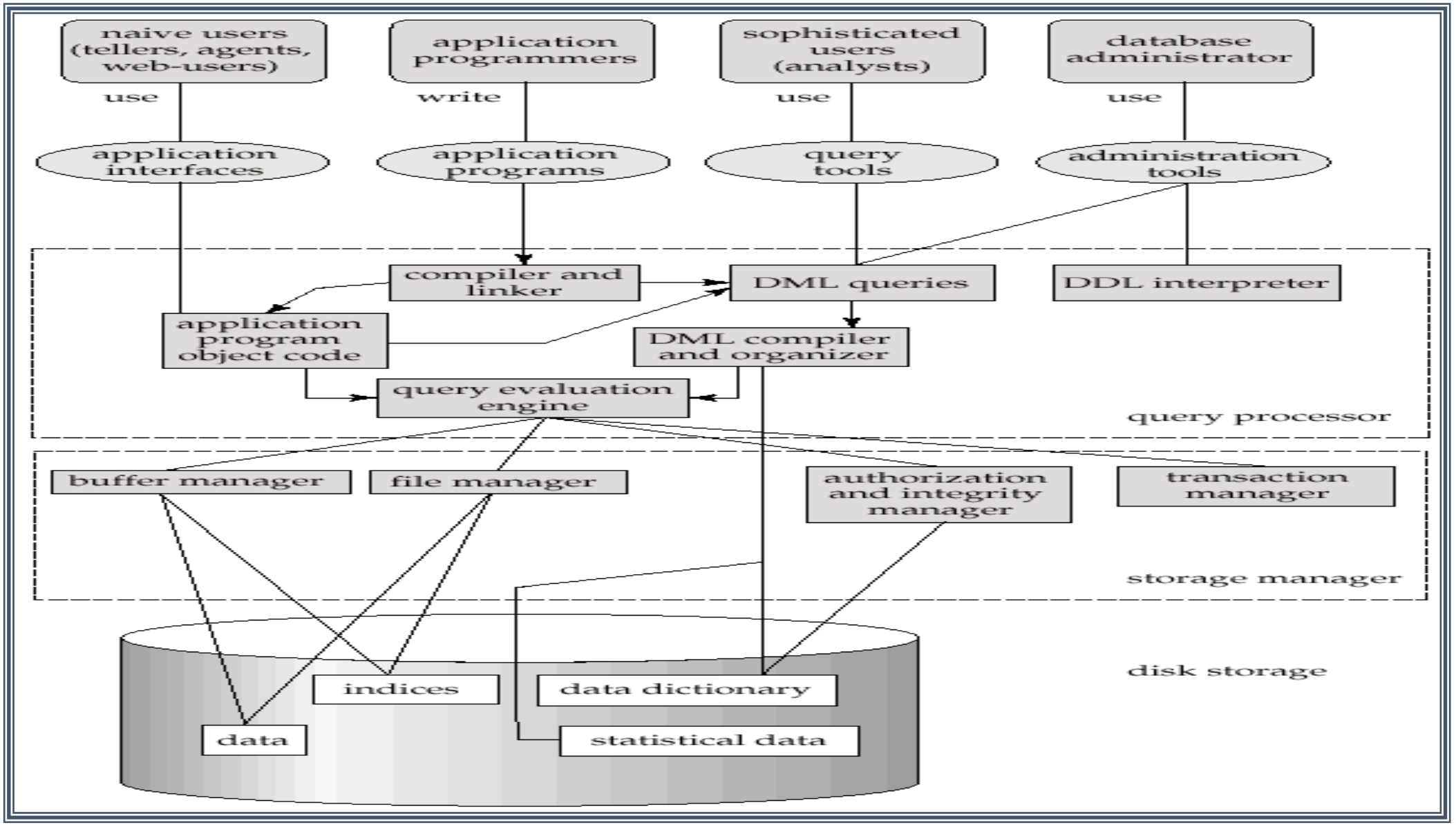
Logical level data independence

- Characteristic of being **able to modify the logical schema without affecting the external schema or application program**.

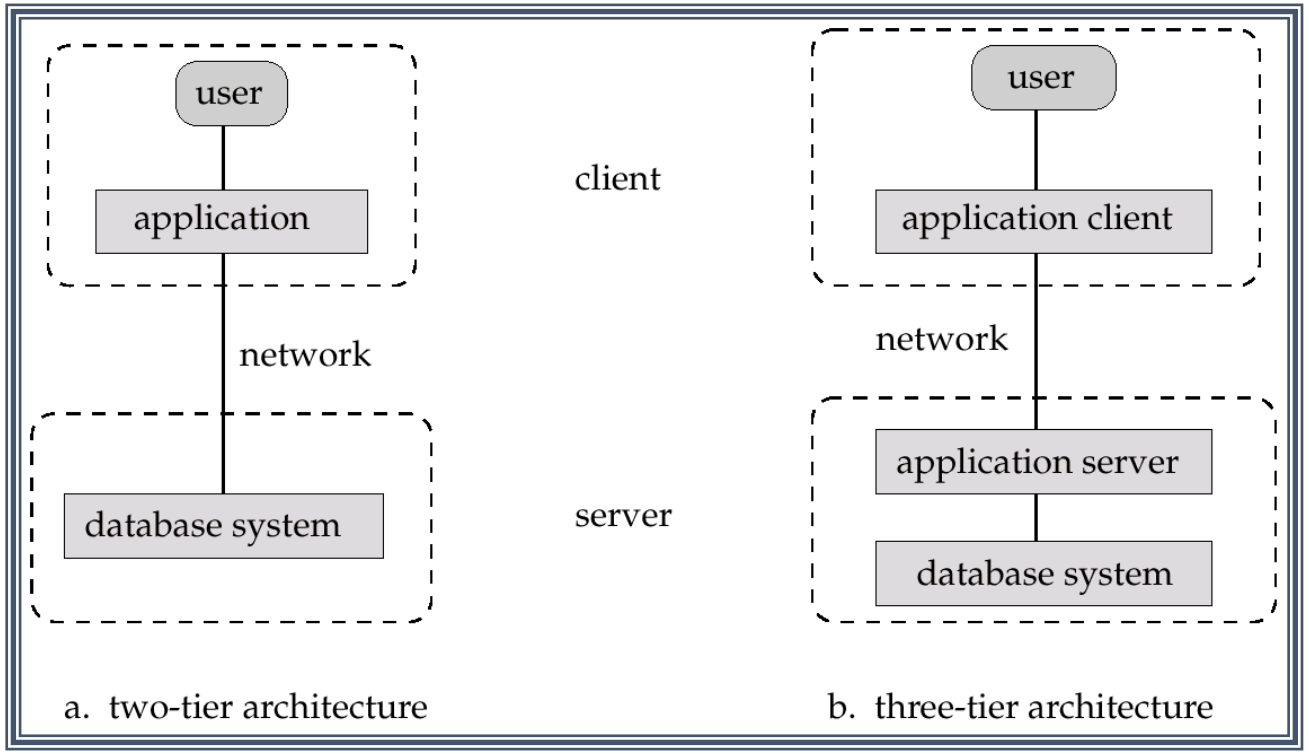
- The user view of the data would not be affected by any changes to the conceptual view of the data.
- These changes may include:
  - **Insertion or deletion of attributes**
  - **Altering table structures entities or relationships to the logical schema etc.**

4. DBMS system architecture

Overall System Structure



Application Architectures



- **Two-tier architecture**: E.g. client programs using ODBC/JDBC to communicate with a database

- **Three-tier architecture**: E.g. web-based applications, and applications built using "**middleware**"

Basic 2-tier Client-Server Architecture

- Specialized Servers with Specialized functions
  - Print server
  - File server
  - DBMS server
  - Web server
  - Email server
- Clients can access the specialized servers as needed

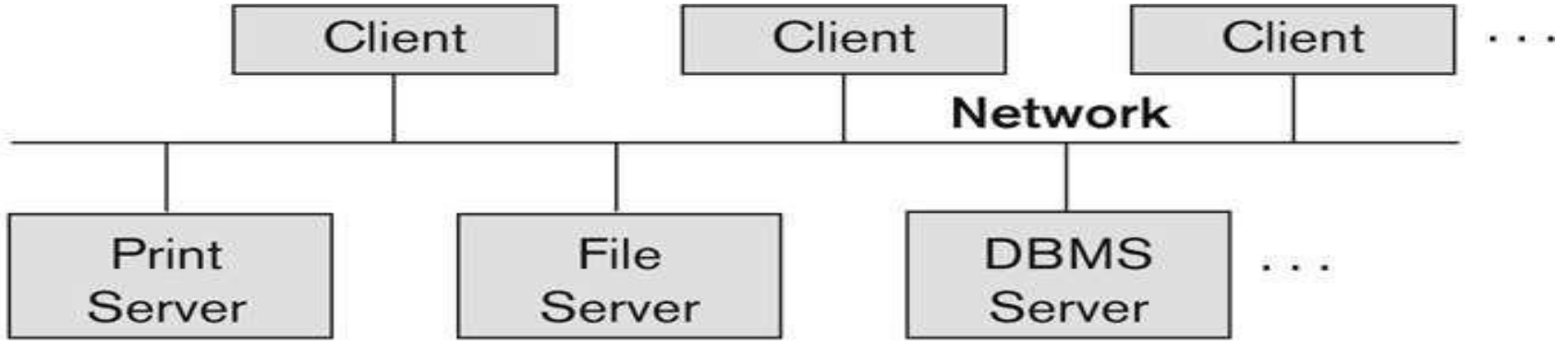
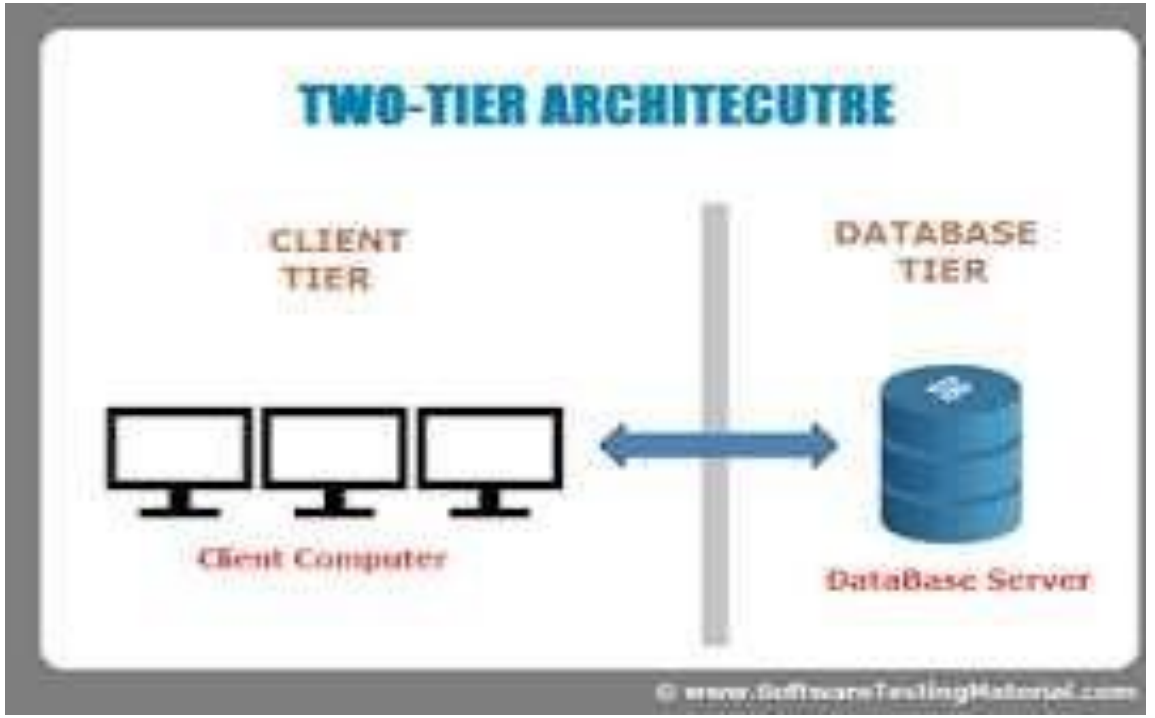


Figure 2.5
Logical two-tier client/server architecture.

Clients

- Provide appropriate interfaces through a client software module
- Clients may be diskless machines or PCs or Workstations with disks
- Connected to the servers via some form of a network.(LAN: local area network, wireless network, etc.)



DBMS Server

- Provides database query and transaction services to the clients
- Applications running on clients utilize an Application Program Interface (**API**) to access server databases via standard interface such as:
  - ODBC: Open Database Connectivity standard
  - JDBC: for Java programming access

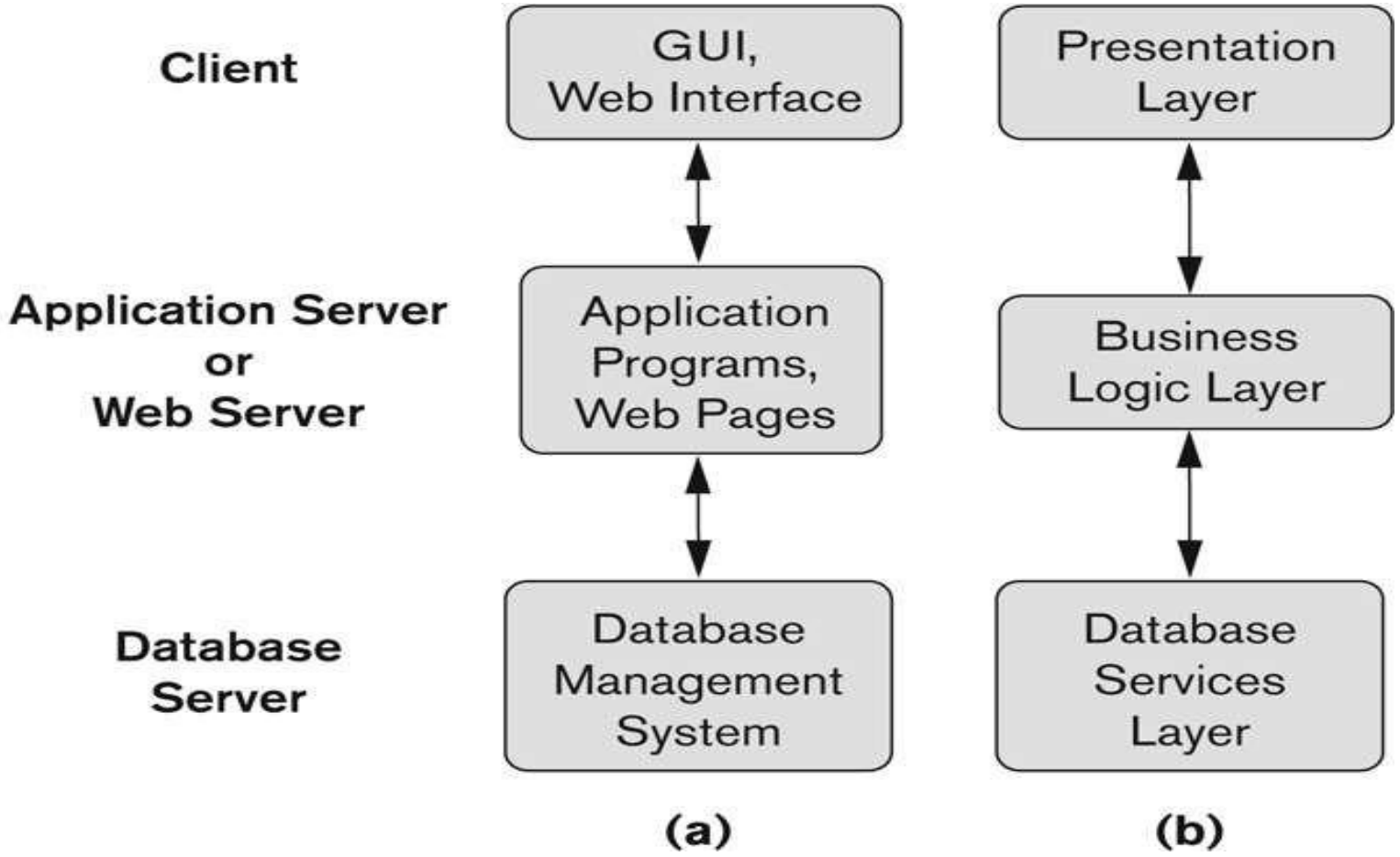
Client and server must install appropriate client module and server module software for ODBC or JDBC

Three Tier Client-Server Architecture

- Common for Web applications
- Intermediate Layer called **Application Server or Web Server:**
  - Stores the web connectivity software and the business logic part of the application used to access the corresponding data from the database server
  - Acts like a agent for sending partially processed data between the database server and the client.
    - Three-tier Architecture Can Enhance Security:
      - Database server only accessible via middle tier
  - Clients cannot directly access database server

Three-tier client-server architecture(contd..)

Figure 2.7
Logical three-tier client/server architecture, with a couple of commonly used nomenclatures.



Database Users

Users are differentiated by the way they expect to interact with the system:

- **Application programmers** – interact with system through DML calls
- **Sophisticated users** – form requests in a database query language
- **Specialized users** – write specialized database applications that do not fit into the traditional data processing framework
- **Naïve users** – invoke one of the permanent application programs that have been written previously
  - E.g. people accessing database over the web, bank tellers, clerical staff

Database administrator

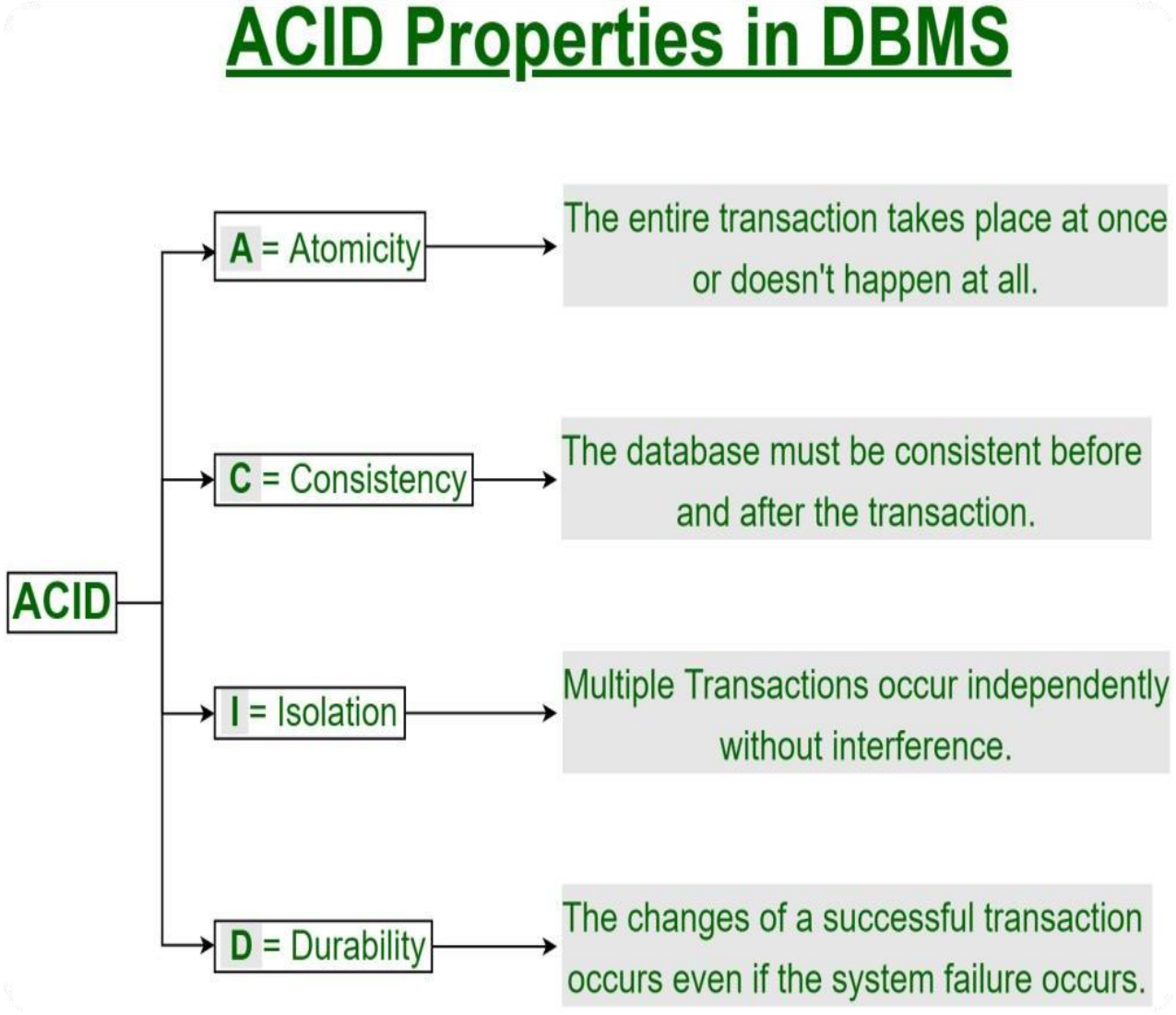
- Individual or person responsible for controlling, maintenance, coordinating, and operation of database management system.
- They are responsible and in charge for authorizing access to database, coordinating, capacity, planning, installation, and monitoring uses and for acquiring and gathering software and hardware resources as and when needed.
- Database administration is major and key function in any firm or organization that is relying on one or more databases. They are overall commander of Database system.

Database Administrator

- Coordinates all the activities of the database system; the database administrator has a good understanding of the enterprise's information resources and needs.
- **Database administrator's Role/duties include:**
  - Schema definition
  - Storage structure and access method definition
  - Schema and physical organization modification
  - Granting user authority to access the database
  - Specifying integrity constraints
  - Acting as liaison with users
  - Monitoring performance and responding to changes in requirements
  - Decides hardware
  - Database design
  - Database implementation
  - Query processing performance
  - Tuning Database Performance

5. ACID properties

- A transaction is a very small unit of a program and it may contain several low-level tasks.
- A transaction in a database system must maintain **A**tomicity, **C**onsistency, **I**solation, and **D**urability – commonly known as **ACID properties** – in order to ensure accuracy, completeness, and data integrity.



Atomicity

- By this, we mean that either the entire transaction takes place at once or doesn't happen at all. There is no midway i.e. transactions do not occur partially.
- It involves the following two operations.
  - **Abort:** If a transaction aborts, changes made to database are not visible.
  - **Commit:** If a transaction commits, changes made are visible.
- Atomicity is also known as the '**All or nothing rule**'.

Atomicity..

- Consider the following transaction T consisting of T1 and T2: Transfer of 100 from account X to account Y.

Before: X : 500	Y: 200
Transaction T	
T1	**T2**
Read (X) X: = X – 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

- If the transaction fails after completion of T1 but before completion of T2, then amount has been deducted from X but not added to Y. This results in an inconsistent database state

Consistency

- This means that integrity constraints must be maintained so that the database is consistent before and after the transaction. It refers to the correctness of a database. Referring to the earlier example,
- The total amount before and after the transaction must be maintained.

  Total **before T** occurs = 500 + 200 = **700**.

  Total **after T** occurs = 400 + 300 = **700**.
- Therefore, database is consistent.
- Inconsistency occurs in case T1 completes but T2 fails. As a result T is incomplete.

Isolation

- This property ensures that multiple transactions can occur concurrently without leading to the inconsistency of database state.
- Transactions occur independently without interference.
- Changes occurring in a particular transaction will not be visible to any other transaction until that change in that transaction is written to memory or has been committed.

Isolation..

- Let X= 500, Y = 500.
- Consider two transactions T and T".

T	T"
Read (X) X: = X*100 Write (X) Read (Y) Y: = Y – 50 Write	Read (X) Read (Y) Z: = X + Y Write (Z)

- Suppose T has been executed till Read (Y) and then T" starts. As a result , interleaving of operations takes place due to which T" reads correct value of X but incorrect value of Y and sum computed by

T": (X+Y = 50,000+500=50, 500)

is thus not consistent with the sum at end of transaction:

T: (X+Y = 50,000 + 450 = 50,450).

- Hence, transactions must take place in isolation and changes should be visible only after they have been made to the main memory.

Durability

- This property ensures that once the transaction has completed execution, the updates and modifications to the database are stored in and written to disk and they persist even if a system failure occurs.

- These updates now become permanent and are stored in non-volatile memory.

- The effects of the transaction, thus, are never lost.

Codd's Rule for Relational DBMS

- E.F Codd was a Computer Scientist who invented the **Relational model for Database** management.

- Based on relational model, the **Relational database was created.**
- **Codd proposed** 13 rules popularly known as **Codd's 12 rules to test DBMS's concept against his relational** model.

- Codd's rule actually define what quality a DBMS requires in order to become a Relational

- Database Management System(RDBMS). Till now, there is hardly any commercial product that follows all the 13 Codd's rules.

Codd's Rule for Relational DBMS

- Codd's rule actually define what quality a DBMS requires in order to become a Relational
- Database Management System(RDBMS). Till now, there is hardly any commercial product that follows all the 13 Codd's rules.

Codd's 12 rules are as follows

- **Rule 0: Foundation rule**

  System must be able to manage database entirely through its relational capabilities

- **Rule 1: Information rule**

  All information(including metadata) is to be represented as stored data in cells of tables. The rows and columns have to be strictly unordered.

- **Rule 2: Guaranteed Access**

  Each unique piece of data(atomic value) should be accessible by : **Table Name + Primary Key(Row) + Attribute(column) .**

- **Rule 3: Systematic treatment of NULL**

  Null has several meanings, it can mean missing data, not applicable or no value. It should be handled consistently. Also, Primary key must not be null, ever. Expression on NULL must give null.

Contd...

- **Rule 4: Active Online Catalog**

  Database dictionary(catalog) is the structure description of the complete **Database and it must be** stored online. The Catalog must be governed by same rules as rest of the database. The same query language should be used on catalog as used to query database.

- **Rule 5: Powerful and Well-Structured Language:**

  One well structured language must be there to provide all manners of access to the data stored in the database.

  Example: **SQL . If the database allows access to the data without the use of this** language, then that is a violation.

- **Rule 6: View Updation Rule**

  All the view that are theoretically updatable should be updatable by the system as well.

Contd…

- **Rule 7: Relational Level Operation**

  There must be Insert, Delete, Update operations at each level of relations. Set operation like Union, Intersection and minus should also be supported.

- **Rule 8: Physical Data Independence**

  The physical storage of data should not matter to the system. If say, some file supporting table is renamed or moved from one disk to another, it should not effect the application.

- **Rule 9: Logical Data Independence**

  If there is change in the logical structure(table structures) of the database the user view of data should not change. Say, if a table is split into two tables, a new view should give result as the join of the two tables. This rule is most difficult to satisfy.

Contd…

- **Rule 10: Integrity Independence**

  The database should be able to enforce its own integrity rather than using other programs. Key and Check constraints, trigger etc, should be stored in Data Dictionary. This also make **RDBMS independent of front-end.**

- **Rule 11: Distribution Independence**

  A database should work properly regardless of its distribution across a network. Even if a database is geographically distributed, with data stored in pieces, the end user should get an impression that it is stored at the same place. This lays the foundation of **distributed database .**

- **Rule 12: Nonsubversion Rule**

  If low level access is allowed to a system it should not be able to subvert or bypass integrity rules to change the data.